boolean Data Type

- For dealing with logical statements, variable of type boolean is supported.
- boolean occupies **1 bit conceptually** (its exact memory representation is JVM-dependent). It stores **only two values**: true or false
  - It is mainly used in: Decision making (if, else)
  - Loops (while, for)
  - Logical operations (&&, ||, !)
- **Unlike C/C++, Java does not allow 1 or 0 to represent true or false. Only the boolean literals true and false are valid.**

Type Casting

- **Type Casting** (or **Type Conversion**) is the process of converting data from **one data type to another data type**.
- It is required when an expression contains **different data types**.

```
int a = 10;
double b = 20.5;
double c = a + b;
```

Here, a is an int and b is a double. Before addition, Java automatically converts a

into double.

- Some type castings are done automatically by a compiler, while others have to be done by the programmer.

- **Types of Type Casting**
- There are **two types** of type casting:
  - **Automatic Type Conversion (Widening)**
    - Done **automatically by the Java compiler**.
    - No explicit casting is required.
  - **Conditions**
    - The source and destination data types must be **compatible**.
    - The destination data type must be **larger** than the source data type.

byte - > Short - > int-> long **float -> double**

char-> int

Casting Incompatible Types

- Explicit Type Casting is the process of converting a larger data type into a smaller compatible data type.
- It is performed manually by the programmer using the cast operator.
- Explicit casting is also called Narrowing Conversion because the destination data type has a smaller range than the source data type.
- Syntax

  variable = (target_type) value;

  double->float>long>int->short>byte

Tasks

- Print area of circle and area of rectangle using double data types.

Scope of variable

- Variable to be declared within any block
- A block is begin with an opening curly brace and ended by a closing curly brace
- A block defines a *scope*.
- Thus, each time you start a new block, you are creating a new scope. A scope determines what objects are visible to other parts of your program.
- Many other computer languages define two general categories of scopes: **global and local.**
- As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.
- Scopes can be nested. For example, each time you create a block of code, you are creating a new, nested scope.

Literal Constants

- A literal is a constant value written directly in the source code.

Type	Example	Description
Integer Literal	10, -25, 1000	Whole numbers
Floating-point Literal	3.14, 2.5f, 45.67	Decimal numbers
Character Literal	'A', '9', '#'	Single character enclosed in single quotes
String Literal	"Hello", "Java"	Sequence of characters enclosed in double quotes
Boolean Literal	true, false	Logical values
Null Literal	null	Represents no object/reference

Symbolic Constants

- A symbolic constant is a variable whose value does not change throughout the program.
- A symbolic constant is declared using the **final** keyword.
- Once initialized, **its value cannot be changed**.
- By convention, symbolic constant names are written in **UPPER_CASE**.
  - Examples: PI, MAX_SIZE, MIN_AGE, INTEREST_RATE.

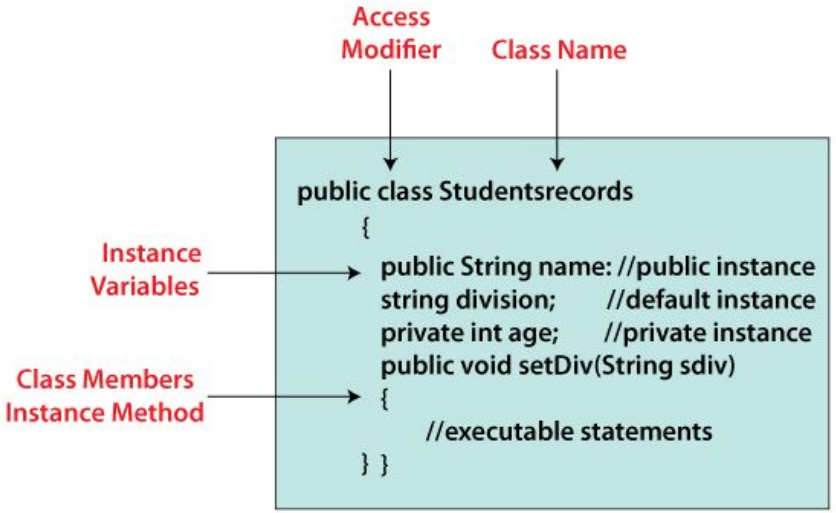
Types of variables

- **Local variables –** inside a method or parameters of a method. **Local variables must be initialized before use**, as they don't have a default value and the compiler **won't allows us to use** an uninitialized value.

In other words, local variables don't have default values.

- **Instance variables –** Declared anywhere in a class but outside of the method. Ea
these variables.
- **Static variable –** declarati
with a keyword static. Onl
JAVA.

Write the differences between local, instance and static variables in



TYPES OF VARIABLES IN JAVA

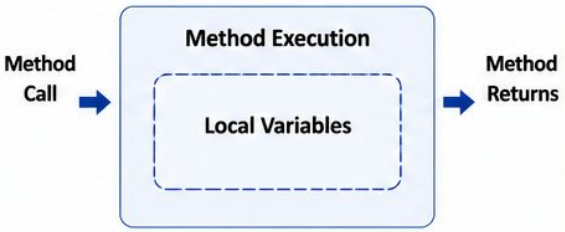
1. LOCAL VARIABLES

- Declared **inside a method** or **parameters of a method**.
- Local variables must be **initialized before use**.
- Don't have **default value**.
- Compiler won't allows us to use an **uninitialized value**.
- Exist only **while the method executes**.

BELONGS TO

Method

MEMORY DIAGRAM



★ Created when the method is called and destroyed when the method returns.

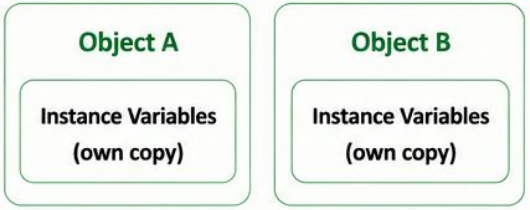
2. INSTANCE VARIABLES

- Declared anywhere in a class but **outside of the method**.
- Each object has its **own copy** of these variables.
- Have **default values**.
- Exist as long as the **object exists**.

BELONGS TO

Object

MEMORY DIAGRAM



★ Every object has its own copy in heap memory.

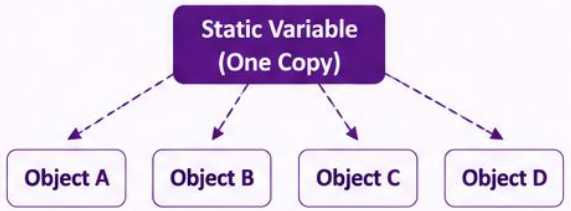
3. STATIC VARIABLES

- Declaration of these variables precede with a keyword **static.**
- Only **one copy** is maintained.
- Shared by all objects of the class.
- Can be accessed using **ClassName.variableName.**
- Exist as long as the **class is loaded** in memory.

BELONGS TO

Class

MEMORY DIAGRAM



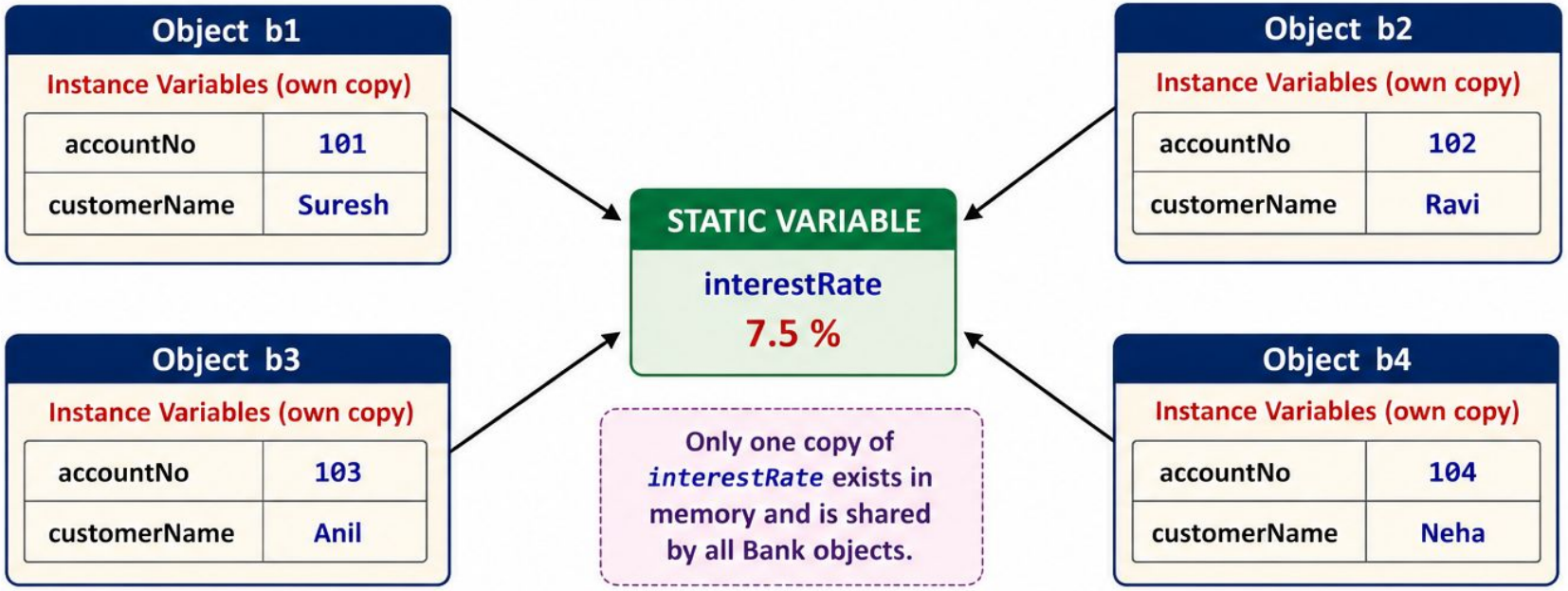
★ Only one copy in method area and shared by all objects.

Key Point: Local variables → Method specific, Instance variables → Object specific, Static variables → Class specific.

Feature	Local Variable	Instance Variable	Static Variable
Declared	Inside a method	Inside a class	Inside a class using static
Belongs to	Method	Object	Class
Default Value	❌ No	✅ Yes	✅ Yes
Memory Created	When method is called	When object is created	When class is loaded
Access	Directly within the method	Using an object	Using class name or directly in the same class

STATIC VARIABLE – Bank Interest Rate Concept

All Bank objects share one static variable (*interestRate*).
Each object has its own copy of instance variables (*accountNo*, *customerName*).



Creating Objects

```
Bank b1 = new Bank(101, "Suresh");
Bank b2 = new Bank(102, "Ravi");
Bank b3 = new Bank(103, "Anil");
Bank b4 = new Bank(104, "Neha");
```

Changing Static Variable

```
Bank.interestRate = 8.0;  // change once
```

Change is reflected in all objects because they share the same static variable.

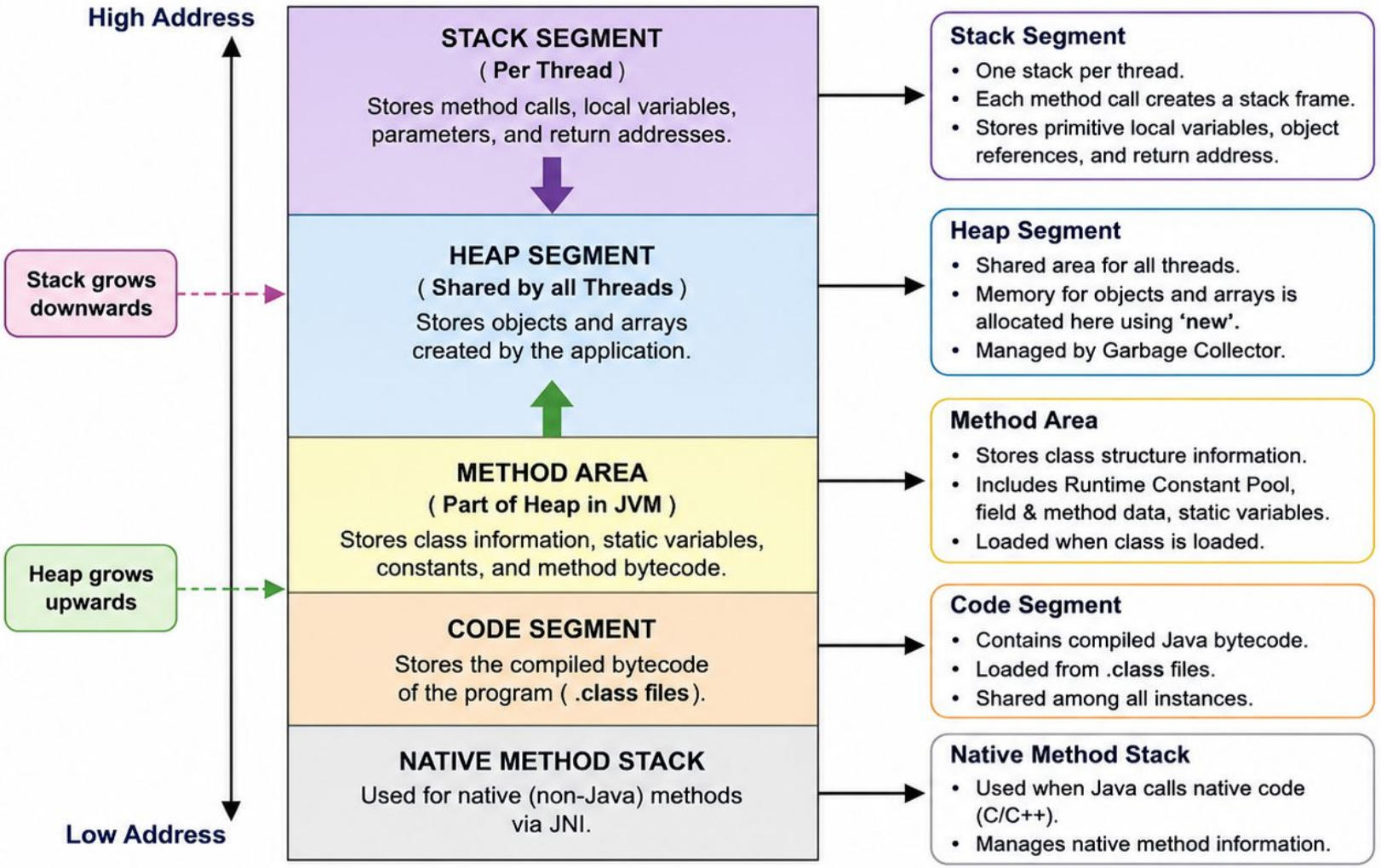
Output After Change

```
101   Suresh   8.0%
102   Ravi     8.0%
103   Anil     8.0%
104   Neha     8.0%
```

Key Point: Instance variables are separate for each object, but static variables are common (one copy) for all objects.

Memory Allocation to a Java Process

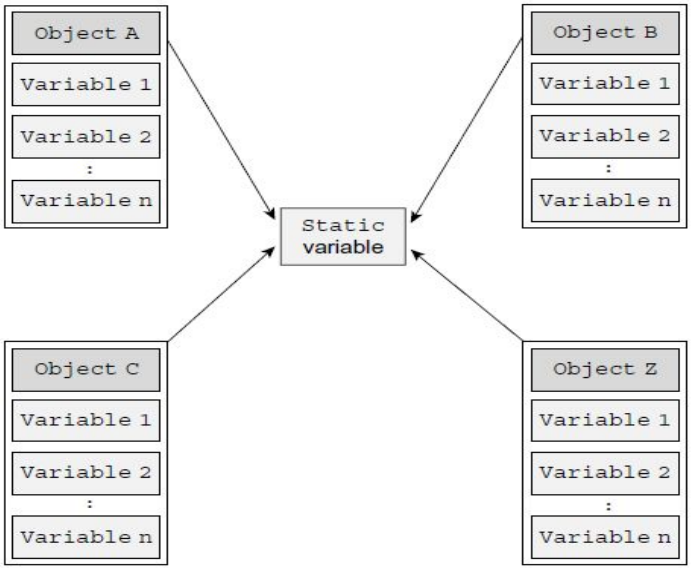
When a Java program runs, the JVM allocates memory to the process in the following regions.



Note : Actual memory layout may differ across JVM implementations, but conceptually a Java process uses these memory regions.

Static Variable

- The variables in a class may be modified by modifier static.
- The non-static variables declared in a class are instance variables.
- Each object of the class keeps a copy of the values of these variables.
- The static variables are class variables.
- The static variables are accessed through class reference, whereas the instance variables are accessed through class object reference.



Attribute Final

- The value of a variable declared final cannot be changed in the program.
- It makes the variable a constant and must be initialized at the time of declaration.
- A few examples of declarations are as:

```
final double PI = 3.14159;
final int M = 900;
final double X = 7.5643;
```



Operators

- **Java operators are symbols used to perform operations on variables and values.**

- Arithmetic operators
- Compound assignment operators
- Increment and decrement operators
- Relational operators
- Boolean logical operators
- Bitwise logical operators
- Shift operators
- Special operators

- If an operator takes only one operand for its operation, it is called unary operator.
- If it takes two operands, it is called binary operator.
- If it takes three operands, it is called ternary operator.

ARITHMETIC OPERATORS IN JAVA

Arithmetic operators are used to perform mathematical operations on numeric values.

Operator	Name	Description	Example	Explanation	Result
+	Addition	Adds two operands	`int a = 10, b = 5;` `int c = a + b;`	c = 10 + 5	15
−	Subtraction	Subtracts right operand from left operand	`int a = 10, b = 5;` `int c = a - b;`	c = 10 - 5	5
*	Multiplication	Multiplies two operands	`int a = 10, b = 5;` `int c = a * b;`	c = 10 * 5	50
/	Division	Divides left operand by right operand	`int a = 10, b = 5;` `int c = a / b;`	c = 10 / 5	2
%	Modulus (Remainder)	Returns remainder after division	`int a = 10, b = 3;` `int c = a % b;`	c = 10 % 3	1

++	Increment	Increases value by 1	`int a = 5;`
			`int b = ++a;`
			`// or a = a + 1;`
				a = 6, b = 6	6

−−	Decrement	Decreases value by 1	`int a = 5;`
			`int b = --a;`
			`// or a = a - 1;`
				a = 4, b = 4	4

Note: In Java, if both operands are integers, division (/) returns integer result (decimal part is discarded). To get decimal result, at least one operand must be float or double.

Increment (++) and Decrement (−−) Operators

Table 3.9 Increment and decrement operators

Operator	*Symbol*	*Code*	*Description*
Pre-increment	++	++x	First increases *x* by unity then evaluates expressions
Post-increment	++	x++	First evaluates expression with current value of *x*, and then, increases *x* by unity
Pre-decrement	--	--x	First decreases *x* by unity then evaluates expression
Post-decrement	--	x--	Evaluates the expression with current value of *x*, and then decreases *x* by unity

```
int x=20, y=10,z;
z=++x + y;
```

z? and x?

```
z=x++ + y;  if x=20, y=10
```

RELATIONAL OPERATORS IN JAVA

Relational operators are used to compare two values. They return boolean result: **true** or **false**.

Operator	Name	Description	Example	Explanation	Result

==	Equal to	Returns true if both operands are equal.	`int a = 10, b = 10;`
			`boolean c = (a == b);`
			`System.out.println(c);`
				a is equal to b
				(10 == 10)
					true

!=	Not equal to	Returns true if both operands are not equal.	`int a = 10, b = 5;`
			`boolean c = (a != b);`
			`System.out.println(c);`
				a is not equal to b
				(10 != 5)
					true

>	Greater than	Returns true if left operand is greater than right operand.	`int a = 10, b = 5;`
			`boolean c = (a > b);`
			`System.out.println(c);`
				a is greater than b
				(10 > 5)
					true

<	Less than	Returns true if left operand is less than right operand.	`int a = 10, b = 5;`
			`boolean c = (a < b);`
			`System.out.println(c);`
				a is less than b
				(10 < 5)
					false

>=	Greater than or equal to	Returns true if left operand is greater than or equal to right operand.	`int a = 10, b = 10;`
			`boolean c = (a >= b);`
			`System.out.println(c);`
				a is greater than or equal to b
				(10 >= 10)
					true

<=	Less than or equal to	Returns true if left operand is less than or equal to right operand.	`int a = 5, b = 10;`
			`boolean c = (a <= b);`
			`System.out.println(c);`
				a is less than or equal to b
				(5 <= 10)
					true



Note: Relational operators always return boolean values: **true** or **false**.

JAVA LOGICAL OPERATORS

Logical operators are used to combine boolean expressions.
They return a boolean value: true or false.

OPERATOR	NAME	BEHAVIOR	EXAMPLE EXPRESSION	RESULT
&&	Short-Circuit AND	Returns true if both operands are true. Skips second if first is false.	(5 > 2) && (3 < 4)	true
||	Short-Circuit OR	Returns true if at least one operand is true. Skips second if first is true.	(2 > 5) || (3 < 4)	true
!	Logical NOT	Unary operator; inverts the boolean value.	!(5 > 2)	false
&	Non-Short-Circuit AND	Returns true if both operands are true. Always evaluates both sides.	isValid & checkNext()	Evaluates both sides
|	Non-Short-Circuit OR	Returns true if at least one operand is true. Always evaluates both sides.	isValid | checkNext()	Evaluates both sides
^	Logical XOR	Returns true if exactly one operand is true (operands must be different).	true ^ false → true	true

Note: && and || are short-circuit operators (more efficient). & and | are non-short-circuit operators (always evaluate both sides).

BITWISE OPERATORS IN JAVA

Bitwise operators perform bit-level operations on integer types (**byte**, **short**, **int**, **long**). They work on the binary representation of numbers.

Operator	Name	Description	Example (int a = 5, b = 3)	Binary Representation	Result

&	Bitwise AND	Returns 1 if both corresponding bits are 1; otherwise 0.	`a = 5, b = 3;`
			`int c = a & b;  // c = 1`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a & b = 0000 0001 (1)`
					1

|	Bitwise OR	Returns 1 if at least one of the corresponding bits is 1; otherwise 0.	`a = 5, b = 3;`
			`int c = a | b;  // c = 7`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a | b = 0000 0111 (7)`
					7

^	Bitwise XOR	Returns 1 if the corresponding bits are different; otherwise 0.	`a = 5, b = 3;`
			`int c = a ^ b;  // c = 6`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a ^ b = 0000 0110 (6)`
					6

~	Bitwise NOT (Complement)	Inverts all the bits (0 becomes 1, 1 becomes 0).	`a = 5;`
			`int c = ~a;  // c = -6`
				`a = 5 = 0000 0000 0000 0000 0000 0101`
				`~a = 1111 1111 1111 1111 1111 1010`
				(In 32-bit signed int, this is -6)
					-6

<<	Left Shift	Shifts bits to the left by specified positions. Rightmost bits filled with 0.	`a = 5;` `int c = a << 1;  // c = 10`	`a = 0000 0101` `a << 1 = 0000 1010 (10)`	10
>>	Right Shift (Signed)	Shifts bits to the right by specified positions. Leftmost bit (sign bit) is extended.	`a = 5;` `int c = a >> 1;  // c = 2`	`a = 0000 0101` `a >> 1 = 0000 0010 (2)`	2
>>>	Unsigned Right Shift	Shifts bits to the right by specified positions. Leftmost bits filled with 0.	`a = 5;` `int c = a >>> 1;  // c = 2`	`a = 0000 0101` `a >>> 1 = 0000 0010 (2)`	2

Note:

- All bitwise operators work only with integral types.
- ~, <<, >>, >>> are unary or binary depending on usage.
- For negative numbers, Java uses 2's complement representation.

Quick Example

```
int a = 5;  // 0000 0101
int b = 3;  // 0000 0011

a & b = 1  (0000 0001)
a | b = 7  (0000 0111)
a ^ b = 6  (0000 0110)

~a    = -6  (1111...1010)
a << 1 = 10  (0000 1010)
a >> 1 = 2   (0000 0010)
a >>> 1 = 2  (0000 0010)
```

JAVA SHIFT OPERATORS: >> vs >>>

Shifts all bits to the right

Understanding Signed Right Shift and Unsigned Right Shift

SIGNED RIGHT SHIFT (>>)

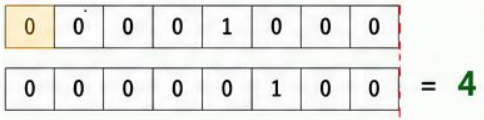
- Copies the sign bit (MSB).
- Preserves the sign of the number.
- Positive remains positive.
- Negative remains negative.

Memory Trick:
>>
= Keep the Sign

Positive Number Example

8 = 00001000

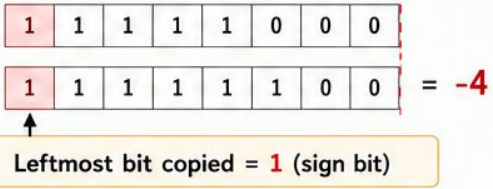
8 >> 1



Negative Number Example

-8 = 11111000

-8 >> 1



UNSIGNED RIGHT SHIFT (>>>)

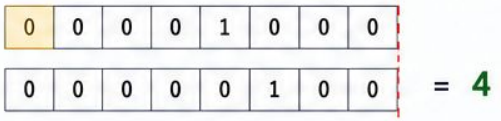
- Always inserts 0 on the left.
- Does NOT preserve the sign.
- Negative numbers become large positive numbers.

Memory Trick:
>>>
= Fill with Zero

Positive Number Example

8 = 00001000

8 >>> 1



Negative Number Example

-8 = 11111000

-8 >>> 1



2'S COMPLEMENT OF 8

2's Complement is a way to represent negative numbers in binary.

Steps: 1) Write the binary number 2) Find 1's complement 3) Add 1

Find 2's Complement of +8 (in 8-bit representation)

STEP 1

Write the binary of +8

0	0	0	0	1	0	0	0
128	64	32	16	8	4	2	1

$+8 = 00001000_2$

STEP 2

Find 1's Complement (invert all bits)

1	1	1	1	0	1	1	1
128	64	32	16	8	4	2	1

1's Complement $= 11110111_2$

STEP 3

Add 1 to the 1's Complement

1	1	1	1	0	1	1	1
							+ 1
1	1	1	1	1	0	0	0

2's Complement of 8 $= 11111000_2$

Verification

8 → 00001000

2's Complement of 8 → 11111000

Add both:

```
  00001000
+ 11111000
-----------
 100000000  (Ignore carry out)
```

Result is 0, which proves it is a correct 2's Complement.

Summary

Positive 8 (8-bit)	:	00001000
1's Complement	:	11110111
Add 1	:	+ 1
2's Complement of 8	:	11111000

ASSIGNMENT OPERATORS IN JAVA



Assignment operators are used to assign a value to a variable.
They combine an operation with assignment.

Operator	Name	Description	Example	Explanation	Result
=	Simple Assignment	Assigns the value on the right to the variable on the left.	`int a = 10;` `a = 25;`	a gets the value 25	a = 25
+=	Add and Assign	Adds right operand to the left operand and assigns the result to the left operand.	`int a = 10;` `a += 5;`	`a = a + 5` `a = 10 + 5`	a = 15
-=	Subtract and Assign	Subtracts right operand from left operand and assigns the result to the left operand.	`int a = 10;` `a -= 4;`	`a = a - 4` `a = 10 - 4`	a = 6
*=	Multiply and Assign	Multiplies left operand with right operand and assigns the result to the left operand.	`int a = 10;` `a *= 3;`	`a = a * 3` `a = 10 * 3`	a = 30
/=	Divide and Assign	Divides left operand by right operand and assigns the result to the left operand.	`int a = 20;` `a /= 4;`	`a = a / 4` `a = 20 / 4`	a = 5
%=	Modulus and Assign	Computes modulus of left operand by right operand and assigns the result to the left operand.	`int a = 20;` `a %= 6;`	`a = a % 6` `a = 20 % 6`	a = 2

Also called as compounded assignment operators.

TERNARY OPERATOR IN JAVA (Conditional Operator)

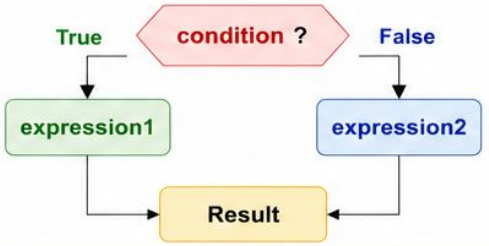
The ternary operator is a shorthand way of writing an if-else statement.

It takes three operands and has the form: `condition ? expression1 : expression2`

Part	Meaning	Description
condition	Test Expression	**Evaluated first.** Must be a boolean expression.
expression1	Value if condition is true	Returned (or evaluated) if condition is true.
expression2	Value if condition is false	Returned (or evaluated) if condition is false.

How it works?

If condition is true,
the result of expression1 is returned.
Otherwise (condition is false),
the result of expression2 is returned.



EXAMPLES

#	Example	Explanation	Result

1	`int a = 10, b = 20;`
	`int max = (a > b) ? a : b;`
	`System.out.println(max);`
		Check if a > b is true?
		Yes (false). So, value of b is returned.
			20

2	`int a = 15, b = 10;`
	`int max = (a > b) ? a : b;`
	`System.out.println(max);`
		Check if a > b is true?
		Yes (true). So, value of a is returned.
			15

3	`int n = -5;`
	`String msg = (n >= 0) ? "Positive" : "Negative";`
	`System.out.println(msg);`
		Check if n >= 0 is true?
		No (false). So, "Negative" is returned.
			Negative

4	`int mark = 75;`
	`String result = (mark >= 40) ? "Pass" : "Fail";`
	`System.out.println(result);`
		Check if mark >= 40 is true?
		Yes (true). So, "Pass" is returned.
			Pass

5	`int x = 7;`
	`int evenOdd = (x % 2 == 0) ? 1 : 0;`
	`System.out.println(evenOdd);`
		Check if x % 2 == 0 is true?
		No (false). So, 0 is returned.
			0

Equivalent if-else Statements

```
int max = (a > b) ? a : b;
```
is same as
```
int max;
if (a > b)
    max = a;
else
    max = b;
```

```
String msg = (n >= 0) ?
"Positive" : "Negative";
```
is same as
```
String msg;
if (n >= 0)
    msg = "Positive";
else
    msg = "Negative";
```

```
int evenOdd = (x % 2 == 0) ? 1 : 0;
```
is same as
```
int evenOdd;
if (x % 2 == 0)
    evenOdd = 1;
else
    evenOdd = 0;
```

Note: Ternary operator is right associative. Use parentheses for complex conditions for better readability.

Points to Remember

- ✓ It improves code readability for simple if-else conditions.
- ✓ Not recommended for complex logic.

TYPE COMPARISON OPERATOR IN JAVA



The **instanceof** operator is used to compare an object with a type (class or interface). It returns **true** if the object is an instance of that type; otherwise it returns **false**.

Operator	Name	Description	Syntax	Example	Result

instanceof	Type Comparison Operator	Checks whether an object is an instance of the specified type.	`object instanceof Type`	`String s = "Hello";`
				`boolean b = s instanceof String;`
				`System.out.println(b);`
					true

EXAMPLES

1

```
Object obj = new String("Java");
boolean r = obj instanceof String;
System.out.println(r);   // true
```

obj refers to a String object.
So, it is an instance of **String**.

2

```
Object obj = new Integer(10);
boolean r = obj instanceof Number;
System.out.println(r);    // true
```

Integer is a subclass of **Number**.
So, it is an instance of **Number**.

3

```
String s = "Java";
boolean r = s instanceof Integer;
System.out.println(r);    // false
```

String is not an instance of **Integer**.
So, result is false.

Points to Remember

- Used with reference variables (objects).
- Used to check the type before type casting.
- Returns boolean (**true**/**false**).

`object instanceof Type`
↓
true or **false**

Precedence and Associativity of Operators

- **Precedence** and **associativity** are two features of Java operators.
- When there are two or more operators in an expression, the operator with the highest priority will be executed first.
  - Ex: 1 + 2 * 5
  - Precedence specifies – Operator with highest priority level
- What if the precedence of all operators in an expression is the same?

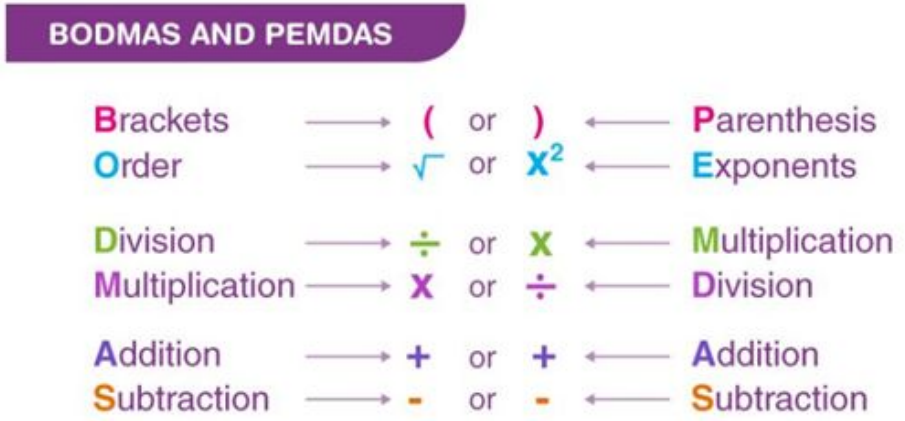
  Ex: 6*4%5 - Precedence is same at same level

  Ans: Associativity comes into the existence

- **Associativity** specifies the order in which operators are executed, which can be left to right or right to left.

```
int a = b = c = 8 ;   // right to left
```

- **BODMAS** rule is a rule in mathematics used to solve the expression involving more operators.





O – Order of means – powers, Square root, exponents ….

Precedence	Operator	Type	Associativity
1 (Highest)	() [] .	Parentheses Array subscript Member selection	Left to Right
2	++ --	Unary post-increment Unary post-decrement	Left to Right
3	++ -- + - ! ~ (type)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
4	* / %	Multiplication Division Modulus	Left to right
5	+ -	Addition Subtraction	Left to right
6	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right

Precedence	Operator	Type	Associativity
7	< <= > >= instanceof	less than less than or equal greater than greater than equal Type comparison (objects only)	Left to right
8	== !=	Relational is equal to Relational is not equal to	Left to right
9	&	Bitwise AND	Left to right
10	^	Bitwise exclusive OR	Left to right
11	|	Bitwise inclusive OR	Left to right
12	&&	Logical AND	Left to right
13	||	Logical OR	Left to right
14	? :	Ternary conditional	Right to left
15 (lowest)	= += -= *= /= %=	Assignment Addition assignment Subtraction assignment Multiplication assignment Division assignment Modulus assignment	Right to left

Assignment Operator

- The associativity of = is from *right to left*.
- On the left of =, we write the name of the variable to which a value is to be assigned, and on its right, we write the value to be assigned.
- The name or identifier on the left is also called ***l*-value** and the value on the right of = is called ***r*-value**.

l-value(name of the identifier) = r-value(identifier or value)

Chain of Assignments:

```
int a,b,c,d;
A=b=c=d=50;
```

Basic Arithmetic Operators

Table 3.7 Basic arithmetic operators

Operation	*Operator*	*Description*
Addition	+	Adds two numbers (binary), changes sign (unary)
Subtraction	–	Subtracts two numbers (binary), changes sign (unary)
Multiplication	*	Multiplies two operands (binary)
Division	/	Divides an operand by another (binary)
Modulus	%	Gives remainder of a division operation. In Java, it can be applied to both integers as well as floating point numbers.

Ex:ArithmeticOperators.java

Combinational Assignment Operators

- In a compound assignment operator, the operations by an operator and assignment are combined in a single operator. As for e.g.
- int n = 8, m = 10;
- n + = m;

Program 3.23: Illustration of combinational operators

```
 1  class Operators2{
 2  public static void main (String args[]){
 3  int a = 4, b = 8, c= 9, d = 12;
 4  int e = 7, f = 10;
 5  b += a;
 6  f -= e;
 7  d /= 3;
 8  c %= a;
 9  e *= 3;
10  System.out.println ("b = " + b );
11  System.out.println ("c = " + c + "\t d = " + d);
12  System.out.println ("e = " + e + "\t f = " + f);
13  }}
```

Ternary Operator

- Ternary operator is also called conditional operator.
- In Java, there is only one ternary operator described by token (?:).
- It takes three operands for its operation.

expression1? expression2 : expression3

- If expression1 is true, then expression2 is executed; otherwise, expression3 is executed.

Relational Operators

Table 3.10 Relational operators

Operator	*Description*	*Code example with if (expression)*
==	Equality	`if (a == b)`
!=	Non-equality	`if (a != b)`
>	Greater than	`if (a > b)`
<	Less than	`if (a < b)`
>=	Greater or equal to	`if (a >= b)`
<=	less or equal to	`if (a <= b)`

Boolean Logical Operators

Table 3.11 Boolean logical operators

Operator	*Description*	*Description*
&	AND	It is true only when both operands in x&y are true.
|	OR	It is false when both operands in x|y are false.
!	NOT	If x = false, then !x = true and vice versa.
^	XOR	False when both x and y in expression x^y are either true or false; otherwise true.
||	OR	Short-circuit form of OR. In x||y, if x = true, then y is not evaluated because the value is anyway true.
&&	AND	Short-circuit form of AND. In x&&y, if x evaluates false, then y is not evaluated, because result is anyway false.

Bitwise Logical Operators

- These operators are applied to individual bits of numbers.
- The bitwise operators are OR, AND, XOR, and bitwise compliment/bitwise NOT operator.

Table 3.14 Bitwise logical operators

Operator	*Description*	*Operator*	*Description*
~	Bitwise complement unary NOT	>>	Right shift filled with previous content called *sign shift.*
|	Bitwise OR	<<	Left shift
&	Bitwise AND	>>=	Right shift assign
^	Bitwise XOR	<<=	Left shift assign
|=	Bitwise OR and assign	>>>	Right shift with zero fill on left
&=	Bitwise AND and assign	>>>=	Right shift, zero fill and assign
^=	Bitwise XOR and assign		

Bitwise Logical Operators

- These operators are applied to individual bits of numbers.
- The bitwise logical operators are AND(&), OR(|), XOR(^), and NOT(~)

Bitwise *OR* (|)

```
0110   - 6
0101   -5
-----
0111   -7
```

Bitwise *AND* (&)

```
0110  - 6
0101   -5
-----
0100  -  4
```

Bitwise XOR (^)

```
0110  - 6
0101 - 5
------
0011  -3
```

Bitwise COMPLEMENT (~)

```
0110 ->  1001
```

Shift operators: << - left shift , >> right shift

Syntax: variable << or >> number_of_digits_to_be_shift

Ex: Bitwise_Operators.java

- **Bitwise Shift Operators**
- Syntax:

  value <operator> <number_of_times>
  - The left side of the expression is the integer that is shifted, and the right side of the expression denotes the number of times that it has to be shifted.
  - Bitwise shift operators are further classified as bitwise left and bitwise right shift operators.

Formatted output with printf() method

- Java provides two ways to format the output displayed on the screen:
  1. **Formatter Class**
  2. **printf() Method** (Most commonly used)
- Formatting means:
  - Right and left justification of text.
  - Controlling the precision of floating-point numbers
  - Displaying integers in decimal, octal, and hexadecimal forms
  - Displaying date and time in different formatsSyntax:
  - Controlling field width and alignment

System.out.printf("Formatting String", variables separated with comma);

System.out.printf("Price :%d",22,"Rupees");

JAVA printf() FORMAT SPECIFIERS

Used in System.out.printf() or System.out.format() to format the output.

Type Variable	Conversion Letter	Formatting String	Description
Integer	d	%d	Decimal (base 10) integer
Integer (Octal)	o	%o	Octal (base 8) integer
Integer (Hexadecimal)	x or X	%x or %X	Hexadecimal (base 16) integer
Character	c (or C)	%c (or %C)	Character (%C prints uppercase of the character)
String	s (or S)	%s (or %S)	String
Floating-point number	f (or F)	%f (or %F)	Decimal floating-point number
Floating-point number (Exponential form)	e (or E)	%e (or %E)	Exponential (scientific) notation
Boolean	b (or B)	%b (or %B)	Boolean value (true / false)
New Line	n	%n	Prints a new line
Percent Sign	%	%%	Prints the % symbol

Examples

- System.out.printf("%d", 25); → 25
- System.out.printf("%x", 255); → ff
- System.out.printf("%.2f", 12.3456); → 12.35
- System.out.printf("%s", "Java"); → Java
- System.out.printf("%e", 123.45); → 1.234500e+02
- System.out.printf("%b", true); → true
- System.out.printf("Hello%nWorld"); → Hello World
- System.out.printf("Discount: 50%% off"); → Discount: 50% off

Ex: O/p of diff types of variables – Format_String.java

Formatting output of integer numbers:

Feature	Description	Syntax	Example
Decimal Integer	Prints the integer in decimal form.	%d	System.out.printf("%d", 163);
Field Width	Specifies the minimum number of character positions reserved for the integer.	%nd	System.out.printf("%10d", 163);
Right Justification	Default alignment. The integer is right-aligned within the specified field width.	%10d	`
Left Justification	Use - after % to left-align the integer.	%-10d	`
Octal Format	Displays the integer in octal (base 8).	%o	243
Hexadecimal Format	Displays the integer in hexadecimal (lowercase).	%x	a3
Hexadecimal (Uppercase)	Displays the integer in hexadecimal (uppercase).	%X	A3

Formatting output of Floating point numbers:

- Field width – the field width is the integer number placed between % sign and conversion character **F or f**
- Ex: Formatting_Floating.java

Format	Description
%f	Floating-point number (6 decimal places by default)
%10f	Right-justified in a field of width 10
%-10f	Left-justified in a field of width 10
%.2f	Displays 2 digits after the decimal point
%10.2f	Field width 10 with 2 decimal places

Formatting output of Strings

- String formatting in Java is done using the %s or %S conversion specifier with the printf() method.

Format	Description	Example Output
%s	Prints the string	Java
%S	Prints the string in uppercase	JAVA
%10s	Right-justified in 10 spaces	Java
%-10s	Left-justified in 10 spaces	Java
%.4s	Prints first 4 characters	Java
%10.4s	Width 10, prints first 4 characters	Java

- Ex: Formatting_String.java

Assignment Questions

- Interpret the differences between local, instance and static(class) variables with suitable examples.
- Write a JAVA program to print factorial of a given number
- Conversion of Decimal to Binary
- Write a JAVA program to generate int and double random numbers using random() method of Math class.
- JAVA program to calculate area of a triagle.
- Develop a program to generate random of 8 integer numbers and calculate its average.

Control Statements

- **Sequential Control Structure:**Statements are executed one after another in the order they appear.
- ***Selection statements:*** These statements give an option of choosing different paths of execution of a program depending on the result of a test condition.
  - The expressions in this group are *if*, *if–else*, and *switch statement*.
- ***Iteration statements:*** These statements allow repeated evaluation of a statement or a block of statements if some condition is satisfied.
  - This group contains statements : *while*, *do–while*, *for*, and *for– each*.
- ***Jump statements:*** These statements make the program control jump to another part of a program in the forward or backward directions with or without a test condition.
  - The statements in this group are *break*, *break label*, *continue*, *continue label*, *and return*.

if Statement

- The code for *if* statement is as follows:

```
if (conditional expression)
        statement;
```

Program 4.1: Illustration of the application of *if* (expression)

```
1    class IfExp {
2     public static void main (String args[])
3    {
4    int n = 51 ;  // integer variable
5     if(n %2 == 0)  // modulus division and checking the remainder
6    System.out.println ("n is an even number");
7     if (n %2 != 0)
8    System.out.println ("n is an odd number.");
9     }
10   }
```

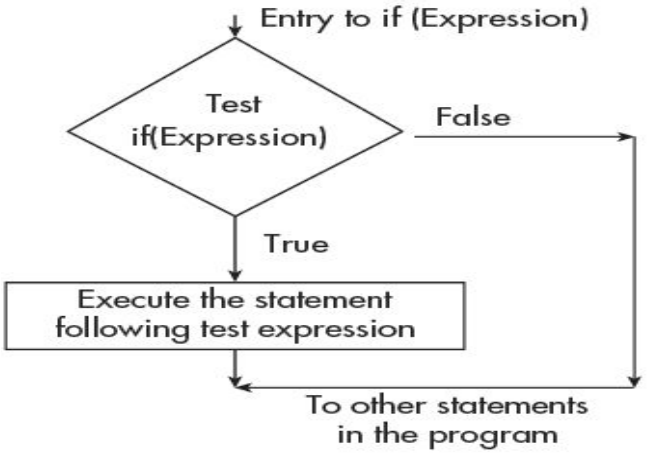
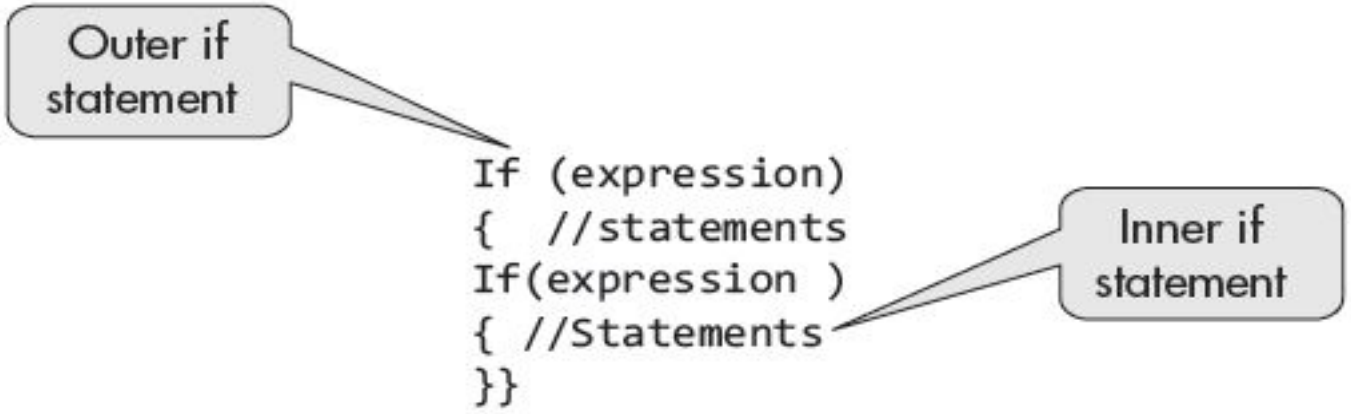


Fig. 4.1 Working of *if* (expression)

Nested if Expressions



If –else statement

- The code for *if –else* statement is as:

```
if (expression)
{
    /* statements */
}
else {
    /* statements */
}
```

Working

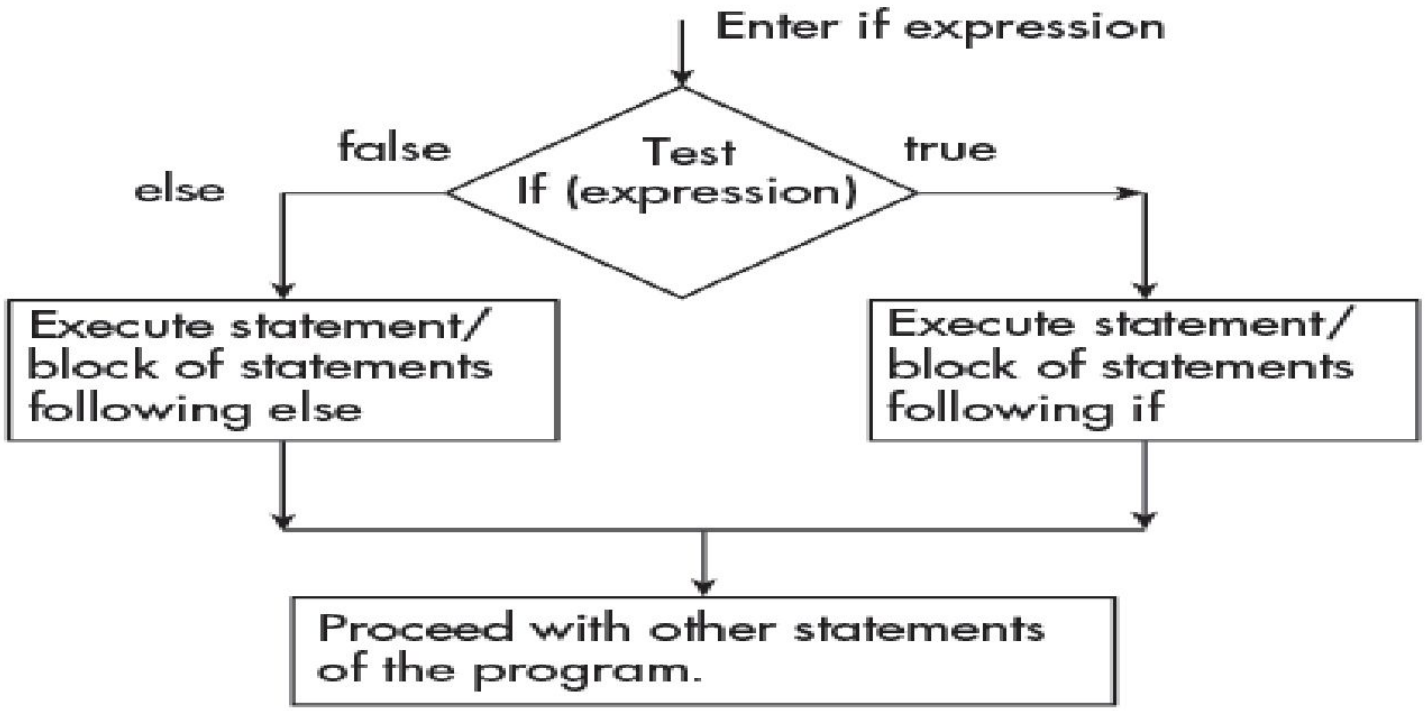


Fig. 4.2 Working of *if–else* expressions

Else-if ladder - if-else chain

if(*condition)*
statement;
else if(*condition)*
statement;
else if(*condition)*
statement;
.
.
.
else
statement;

The **if statements are executed from the top down. As soon as one of the conditions** controlling the **if is true, the statement associated with that if is executed, and the rest of** the ladder is bypassed. If none of the conditions is true, then the final **else statement will be** executed.

Ternary Operator ?:

- Its code is as follows:

  Expression1 ?Expression2 :Expression3
- The first expression is the test condition.
- If it evaluates true, the Expression2 is executed; otherwise Expession3 is executed

Switch Statement

```
switch (expression)
{
    case value1:
      // statement sequence
      break;
case value2:
      // statement sequence
      break;
      .
    case valueN :
      // statement sequence
      break;
default:
    // default statement
    sequence
}
```

1. For versions of Java prior to JDK 7, *expression must be of type* ***byte, short, int, char, or an*** enumeration.
2. Beginning with JDK 7, *expression can also be of type* **String**.

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- The break and default keywords are optional

Nested Switch Statement

```
switch(count)
{
   case 1:
     switch(target) { // nested switch
     case 0:
          System.out.println("target is zero");
          break;
     case 1: // no conflicts with outer switch
          System.out.println("target is one");
          break;
   }
   break;
   case 2:
   .
   .
   .
```

while

```
while(condition)
{
   // body of loop
}
```

```
do {
     // body of loop
}while(condition);
```

```
class WhileSt {
    public static void main(String args[])
    {
        int n = 10;
        while(n > 0) {
            System.out.println("tick " + n);
            n--;
        }
    }
}
```

```
class DoWhileSt
{
    public static void main(String args[]) {
        int n = 10;
        do {
            System.out.println("tick " + n);
            n--;
        } while(n > 0);
    }
}
```

while Expression

- The while loop may be coded as:

  while (test expression)

  statement;

- If the *while* test expression evaluates true, the statement following the expression is executed.

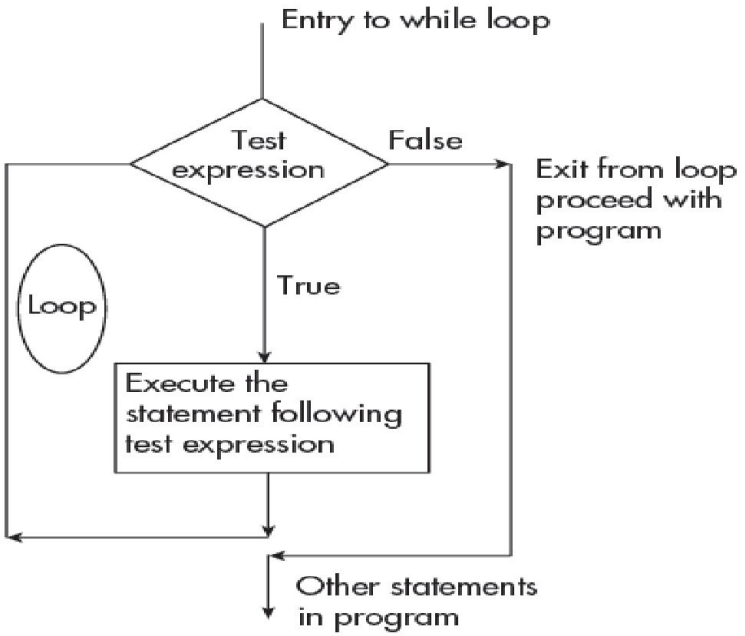


Fig. 4.5 Working of *while* loop

Program 4.8: Illustration of while loop

```
1   class WhileExp {
2    public static void main (String args[]) {
3   int n = 1, sum = 0;
4   while (n <=10) // testing the condition
5    {sum += n; // computing the sum
6    n = n + 1;} // increments the value of variable
7   System.out.println ("sum = " + sum);
8    }
9   }
```

do while Loop

- This loop is similar to *while* loop except for the fact that the test expression is placed at the end.
- The statement to be executed is placed between *do* and *while* expression.

  Code is:

```
do
        Statement;
while (test expression);
```

do while Loop

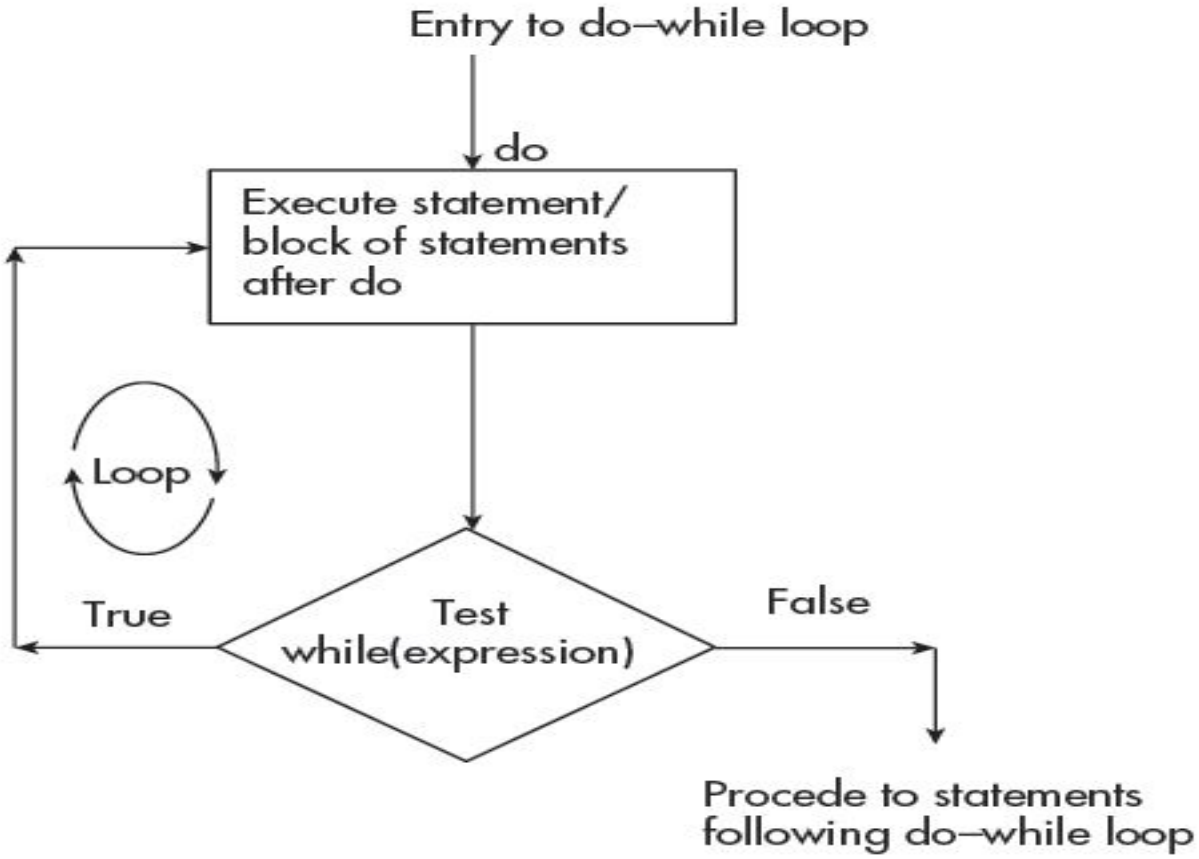


Fig. 4.6 Working of *do–while* loop

© Oxford University Press 2018. All rights reserved.

endless while loop

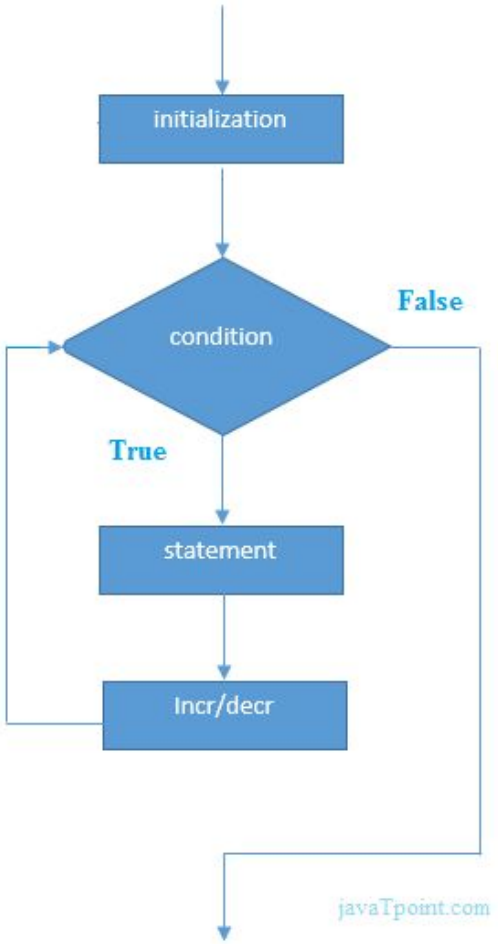
- The endless while loop may be coded as:

```
while (true) {
    // do something
}
 do {
    // do something
} while(true);
```

For loop

- The *for* loop is the most versatile loop when the start and end points of the loop are known.
- It is an entry controlled loop

```
for(initialization; condition;
   incr/decr)
{
   //code to be executed
}
```



Nested for loop

```
for (int i = 0; i < 8; i++)
    { statement;                          // Outermost for loop
      ---------
  for(int j = 0; j < 10; j++)
       { statement;                       // Middle for loop
         ---------
      for (int k = 0; k < 12; k++)
            { statement;                  //innermost
              --------                    // for loop
               --------    }
               statement;
                -------    }
                statement;
                --------
                --------
                           }
```

Fig. 4.8 Nested *for* loops

The for-each loop

- The for-each loop is used to traverse array or collection in java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.
- It works on elements basis not index. It returns element one by one in the defined variable
- Syntax:

```
for(data_type var:array)
{
 //code to be executed
}
```

```
public class ForEachExample
{
  public static void main(String[] args)
{
        int arr[]={12,23,44,56,78};
        for(int i:arr)
        {
            System.out.println(i);
        }
    }
}
```

Nested for-each for loop

```
public class NestedForEach
{
   public static void main(String[] args)
   {
      String[][] str = {{"Amaravati","Vij","Gnt"},
{"Hyd","WL","Khammam"}};

      for (String[] s : str)
      {
         for(String c:s)
         {
      System.out.println(c);
          }
      }
   }
}
```

Break Statement

- This is used for exiting from a loop and control goes to the end of the current loop.

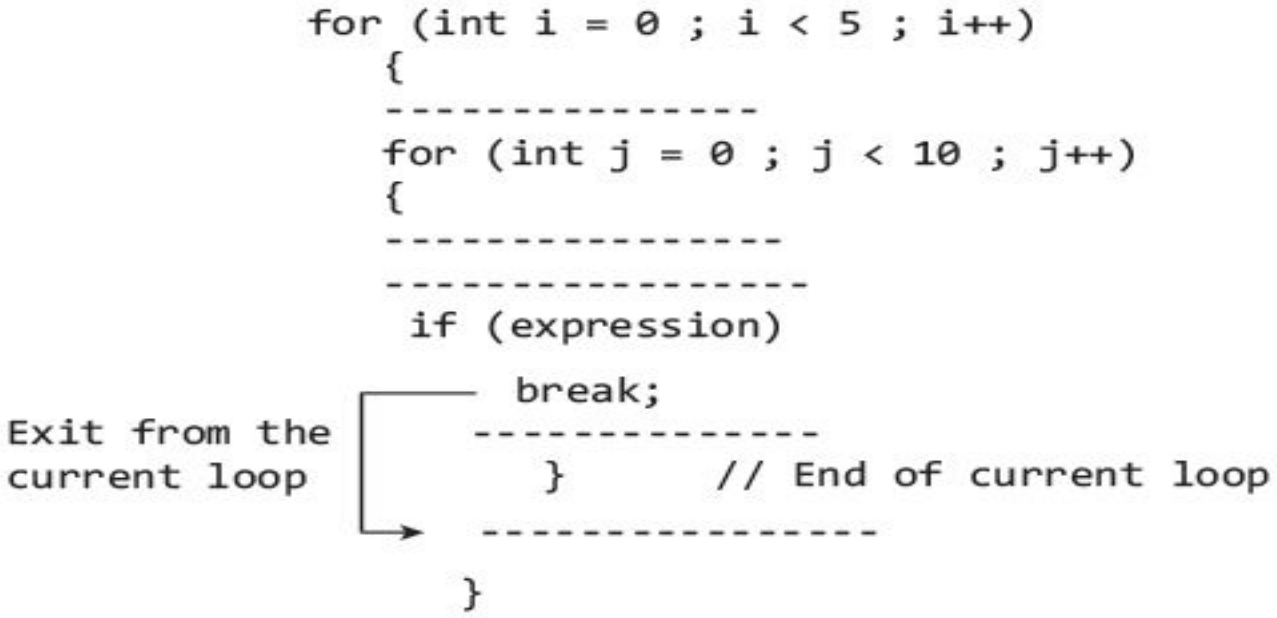
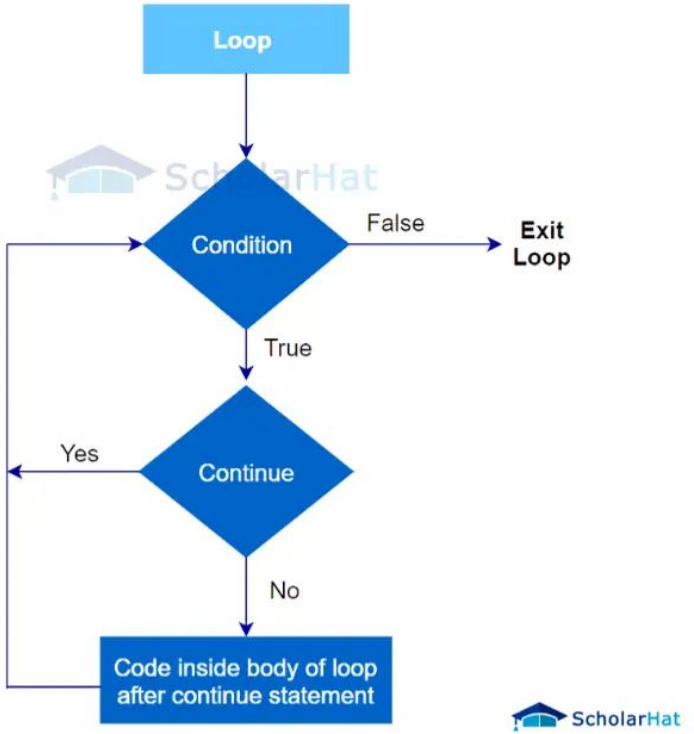


Fig. 4.9 Illustration of *break;* statement

Continue statement

- The continue statement is used to skip the current iteration of any loop and bring the control to the beginning of the iteration.



JUMP STATEMENTS WITH LABEL

Labels can be used with break or continue to transfer control to the labeled outer loop.

3. break with label

```
outer:
  for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
      if (j == 2)
          break outer;  // exit outer loop completely
      System.out.println("i = " + i + ", j = " + j);
    }
  }
```

How it works

When `j == 2`, `break outer;` terminates the labeled outer loop immediately.

4. continue with label

```
outer:
  for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
      if (j == 2)
          continue outer;  // skip to next i
      System.out.println("i = " + i + ", j = " + j);
    }
  }
```

How it works

When `j == 2`, `continue outer;` skips the remaining statements of the current outer iteration and moves to the next value of `i`.

Key Points

- **break label;**
  - Terminates the labeled loop (outer loop here).
  - Control moves to the statement after the labeled loop.
- **continue label;**
  - Skips the remaining part of the current iteration of the labeled loop.
  - Control moves to the next iteration of the labeled loop.

Break Statement with Label

- Break with a label statement is similar to goto statement in C++, but the implementation is different.

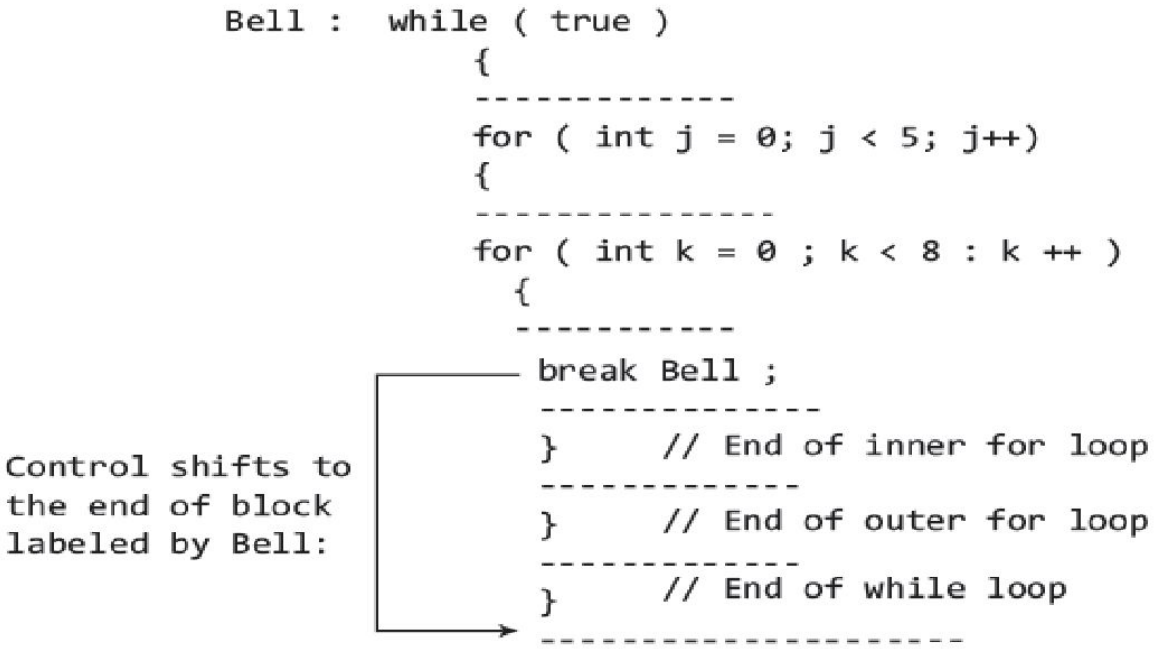


Fig. 4.10 Illustration of break statement with a label

Continue with label

- We can **use** a **labelled continue statement** to **continue** the outermost loop.

Questions

- Diff between while, do-while and for loops
- Diff between break and continue
- To check the given number is even or odd
- To check the given number is prime or not
- Even numbers – with in a specific range
- Prime numbers with in a specific range
- Printing Fibonacci numbers up to n.
- To check the given number is palindrome or not
- To check whether the given number is Armstrong or not.(an Armstrong number is a number that equals the sum of its individual digits, each raised to the power of the number of digits.)